SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)
Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Kanchamreddy Manoj Kumar Reddy certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Manoj

QUESTION 1

(A) Error Identification:

- Line 5 (for(i = 0; i <= n; i++)) – The loop condition should be i < n, not i <= n. Using <= causes the loop to access arr[n], which is outside the valid array bounds and may lead to undefined behavior.
- Line 12 (int arr[5] = {1,2,3,4,5}) – Missing semicolon (;) at the end of the statement, resulting in a syntax error.
- Line 15 (total = sumArray(arr);) – The function sumArray() is defined to accept two parameters (arr and n), but only one argument is passed. The correct call is sumArray(arr, 5);.
- Line 18 (printf("Correct Sum\n")) – Missing semicolon (;) at the end of the printf() statement, causing a syntax error.
- Line 22 (int c = a / b;) – Variable b is initialized to 0, so dividing a by b results in a division-byzero runtime error.
- Line 25 (*p = 20;) – Pointer p is declared but not initialized. Dereferencing an uninitialized pointer writes to an invalid memory location, causing undefined behavior or a segmentation fault.

(B) Classification of Errors

- Line 5 (for(i = 0; i <= n; i++)) – Logical Error. The loop condition is incorrect (<= instead of <), causing the program to access memory outside the array bounds.
- Line 12 (int arr[5] = {1,2,3,4,5}) – Syntax Error. A semicolon (;) is missing at the end of the declaration.
- Line 15 (total = sumArray(arr);) – Semantic Error. The function sumArray() is called with an incorrect number of arguments.
- Line 18 (printf("Correct Sum\n")) – Syntax Error. A semicolon (;) is missing after the printf() statement.
- Line 22 (int c = a / b;) – Runtime Error. Since b is 0, the statement causes a division-by-zero error during execution.
- Line 25 (*p = 20;) – Runtime Error. The pointer p is not initialized before dereferencing, which may cause a segmentation fault.
- Lexical Error – No lexical error is present in the program. All keywords, identifiers, operators, and symbols are valid C language tokens.

(C) Compilation Behavior

Errors Detected During Compilation:

- Missing semicolon after array declaration
- Missing semicolon after printf()

- Incorrect number of arguments in sumArray()

These errors stop compilation.

Errors Detected During Execution:

If compilation succeeds after fixing syntax errors:

- Array out-of-bounds (i <= n)
- Division by zero
- Uninitialized pointer dereference

These errors occur only during program execution.

Why Early Errors Prevent Later Error Detection:

The compiler processes the source code in phases.

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Code Generation

If syntax errors exist, parsing fails and semantic analysis does not proceed. Therefore, runtime-related problems such as division by zero or invalid pointer access cannot be detected until compilation succeeds.

(D) Corrected Program: #include <stdio.h> int sumArray(int arr[], int n)

```
{     int i, sum = 0;
for(i = 0; i < n; i++)
   {        sum = sum +
arr[i];
   }
   return sum;
} int
main()
{
   int arr[5] = {1, 2, 3, 4, 5};
int total;     total =
```

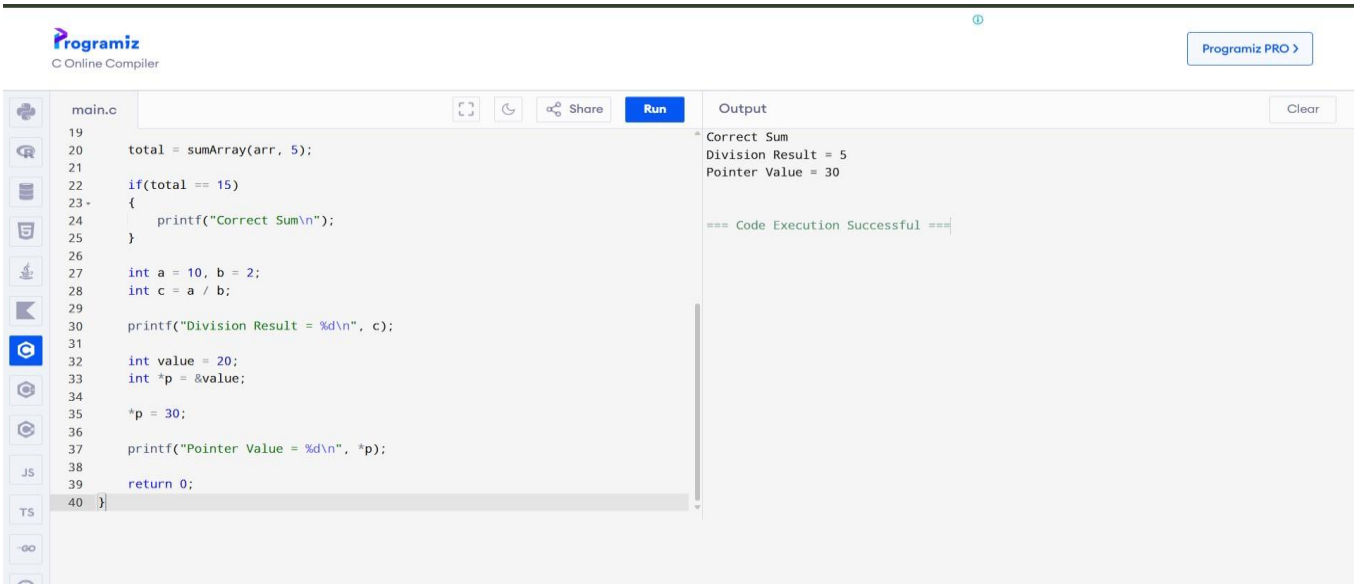
```
sumArray(arr, 5);     if(total
== 15)
   {
      printf("Correct Sum\n");
   }     int a = 10, b = 2;     int c = a /
b;     printf("Division Result = %d\n",
c);     int value = 20;     int *p =
&value;
   *p = 30;     printf("Pointer Value =
%d\n", *p);     return 0;
}
```

Output:

Correct Sum

Division Result = 5

Pointer Value = 30



(E) Impact Analysis:

- Off-by-one loop (line 4): reads arr[n], which is outside the array - this can return garbage values or crash the program, and gives a wrong sum even when it doesn't crash.
- Division by zero (line 17): integer division by zero is undefined behaviour in C and typically terminates the program with a runtime exception (SIGFPE).

- Uninitialised pointer (line 18-19): *p = 20 writes to whatever address p happens to hold, which can crash the program or silently corrupt unrelated memory, making the bug hard to trace. In general: improper pointer usage causes undefined behaviour because the program writes/reads memory it does not own; division by zero causes an immediate runtime crash; infinite loops cause the program to hang and consume CPU indefinitely, so the system becomes unresponsive until it is forcibly terminated.

(F) Advanced Compiler Perspective:

1. Better Error Messages

The compiler should:

- Highlight the exact line and column.
- Suggest possible fixes.
- Display expected syntax.

Example

Instead of Syntax

Error display

Line 12: Missing ';' after array declaration.

Suggestion: Add ';' at the end of the statement.

2. Early Detection of Runtime Errors

Compilers can perform static analysis to detect:

- Division by zero
- Null pointer dereference
- Array index out of bounds
- Use of uninitialized variables

This warns programmers before execution.

3. Compiler Enhancements for Efficient Debugging:

Enhancement 1 – Static Code Analysis:

-

  Detect logical mistakes before execution.

- Warn about unreachable code and memory issues.

Enhancement 2 – Intelligent Error Recovery:

- Continue compilation after minor syntax errors.
- Report multiple errors in one compilation instead of stopping at the first error.

Enhancement 3 – Warning System:

Generate warnings such as:

- Variable declared but not used.
- Possible null pointer access.
- Possible division by zero.
- Array index may exceed bounds.

QUESTION 2:

(A) Error Identification:

- Line 4 (1num) – Invalid identifier. Identifiers cannot begin with a digit.
- Line 5 (rate@) – Invalid identifier. The symbol @ is not allowed in identifiers.
- Line 6 ('abc') – Invalid character constant. A character literal can contain only one character.
- Line 7 (50$20) – Invalid token. The symbol $ is not a valid arithmetic operator in C.
- Line 8 (num#1) – Invalid identifier. The symbol # is not permitted in identifiers.
- Line 9 (%count) – Invalid identifier. An identifier cannot begin with %.
- Line 11 (printf("Welcome to Compiler Design);) – Unterminated string literal. The closing double quotation mark (") is missing.
- Line 13 (0xZ12) – Invalid hexadecimal constant. Z is not a valid hexadecimal digit.
- Line 14 (12.3.4) – Invalid floating-point constant. A numeric constant cannot contain more than one decimal point.
- Line 16 ("Hello;) – Unterminated string literal. The closing double quotation mark is missing.
- Line 20 (invalid-id) – Invalid identifier. The hyphen (-) is treated as the minus operator and cannot be used in identifiers.
- Line 22 (5..6) – Invalid floating-point constant. Multiple decimal points are not allowed.
- Line 23 (10@5) – Invalid token. The symbol @ is not a valid operator in C.

-

- Line 25 ('xy') – Invalid character constant. A character literal can contain only one character.

  Line 26 (3value) – Invalid identifier. Identifiers cannot start with a digit.

(B) Error Classification:

- Line 4 (1num) – Identifier Rule Violation. Identifiers must begin with a letter or underscore (_).
- Line 5 (rate@) – Identifier Rule Violation. Identifiers can contain only letters, digits, and underscores.
- Line 6 ('abc') – Character Literal Rule Violation. A character literal must contain exactly one character.
- Line 7 (50$20) – Operator/Symbol Rule Violation. $ is not a valid operator in C.
- Line 8 (num#1) – Identifier Rule Violation. # is not allowed inside identifiers.
- Line 9 (%count) – Identifier Rule Violation. Identifiers cannot begin with %.
- Line 11 ("Welcome to Compiler Design) – String Literal Rule Violation. Every string literal must end with a closing double quote.
- Line 13 (0xZ12) – Constant Formation Rule Violation. Hexadecimal constants can contain only 0–9 and A–F (or a–f).
- Line 14 (12.3.4) – Constant Formation Rule Violation. A floating-point constant can have only one decimal point.
- Line 16 ("Hello;) – String Literal Rule Violation. Missing closing double quote.
- Line 20 (invalid-id) – Identifier Rule Violation. Hyphen (-) is not allowed in identifiers.
- Line 22 (5..6) – Constant Formation Rule Violation. Floating-point numbers cannot contain multiple decimal points.
- Line 23 (10@5) – Operator/Symbol Rule Violation. @ is not a valid operator.
- Line 25 ('xy') – Character Literal Rule Violation. Character literals must contain only one character.
- Line 26 (3value) – Identifier Rule Violation. Identifiers cannot start with digits.

(C) Token Correction:

- Line 4: 1num → num1
- Line 5: rate@ → rate
- Line 6: 'abc' → 'a'
- Line 7: 50$20 → 50 + 20

-

- Line 8: num#1 → num1

  Line 9: %count → count
- Line 11: "Welcome to Compiler Design → "Welcome to Compiler Design"
- Line 13: 0xZ12 → 0xA12
- Line 14: 12.3.4 → 12.34
- Line 16: "Hello; → "Hello"
- Line 20: invalid-id → invalid_id
- Line 22: 5..6 → 5.6
- Line 23: 10@5 → 10 + 5
- Line 25: 'xy' → 'x'
- Line 26: 3value → value3

CORRECT CODE:

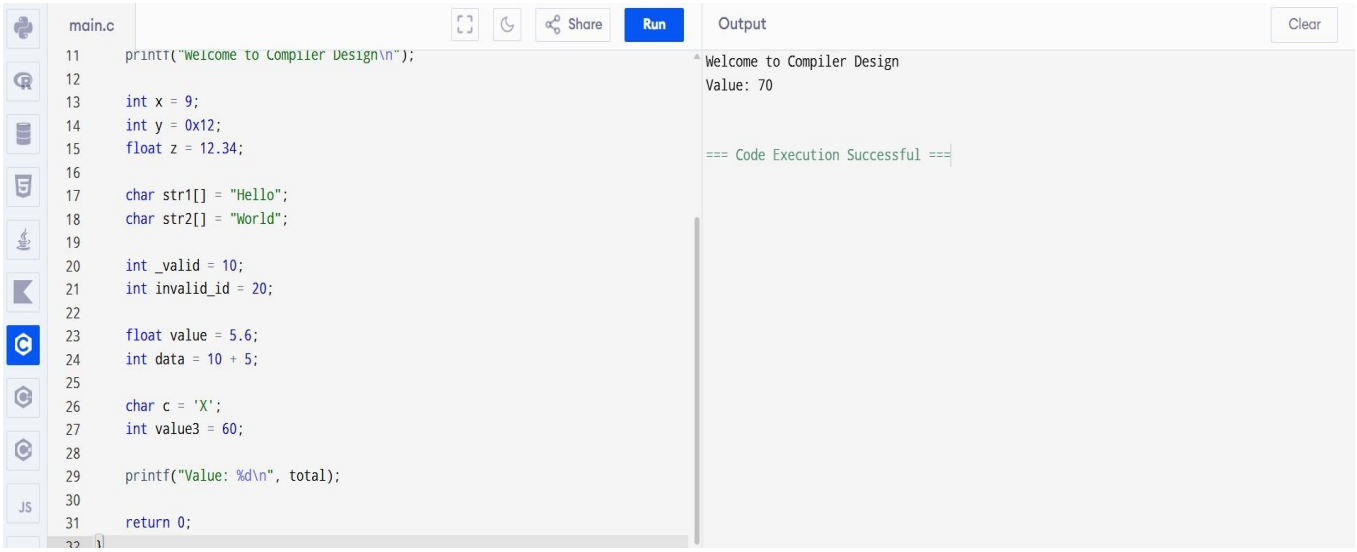
```
#include <stdio.h> int main() {     int num1
= 100;     float rate = 25.5;     char ch = 'A';
int total = 50 + 20;     int num1_copy = 30;
int count = 40;     printf("Welcome to
Compiler Design\n")
   int x = 9;     int y =
0x12;     float z = 12.34;
char str1[] = "Hello";
char str2[] = "World";
int _valid = 10;     int
invalid_id = 20;     float
value = 5.6;     int data
= 10 + 5;     char c =
'X';
```

int value3 = 60;

printf("Value: %d\n", total);

return 0;}

OUTPUT:



(D) Compiler Behavior Analysis:

(i) How does the lexical analyzer process these errors?

The lexical analyzer (scanner) reads the source code character by character and groups them into valid tokens such as identifiers, keywords, constants, operators, and literals. If it encounters an invalid token, it reports a lexical error indicating the line number and the invalid token.

(ii) Does the compiler stop immediately?

- For serious lexical errors (such as unterminated string literals), the compiler may stop because it cannot correctly identify the remaining tokens.
- For minor lexical errors (such as invalid identifiers), most modern compilers report the error and continue scanning using error recovery techniques, allowing multiple errors to be detected in a single compilation.

(E) Advanced Thinking:

(i) DFA (Finite Automaton) Rule to Detect Identifiers Starting with Digits

A simple rule for identifier recognition is:

- The first character must be a letter (A–Z or a–z) or an underscore (_).
- The remaining characters may be letters, digits (0–9), or underscores (_).

- If the first character is a digit, the lexical analyzer immediately reports "Invalid Identifier".

(ii) Suggestions to Improve Error Messages

- Display the exact line number and column number where the error occurs.
- Clearly highlight the invalid token in the source code.
- Provide a suggested correction whenever possible (e.g., 3value → value3).
- Explain the violated lexical rule in simple language suitable for beginners.
- Continue scanning after minor errors so that multiple errors are reported in a single compilation, making debugging easier.

QUESTION 3

(A) Identify Errors:

- Line 4 (return a / b) – Missing semicolon (;) after the return statement, causing a syntax error.
- Line 8 (int #value = 20;) – Invalid identifier. The symbol # is not allowed in identifiers, resulting in a lexical error.
- Line 9 (float num = 15.5) – Missing semicolon (;) at the end of the declaration.
- Line 12 (result = divide(10);) – Function divide() requires two arguments, but only one argument is passed.
- Line 14 (if(result = 5)) – Assignment operator (=) is used instead of the comparison operator (==). This is a logical/semantic error.
- Line 15 (printf("Result is 5\n")) – Missing semicolon (;) after the printf() statement.
- Line 20 (int z = x / y;) – Division by zero because y is initialized to 0.
- Line 23 (printf("%d\n", arr[4]);) – Array index 4 is out of bounds. Valid indices are 0 to 3.
- Line 26 (*ptr = 25;) – Pointer ptr is declared but not initialized before dereferencing, causing invalid memory access.
- Line 29 (total = x + y * result + 0;) – The expression contains the redundant operation + 0, which is unnecessary and can be optimized.
- Line 31–33 (while(x < 20)) – Infinite loop because the value of x is never updated inside the loop.

(B) Classify by Compiler Phase:

- Line 8 (int #value = 20;) – Lexical Error. # is an invalid character in an identifier.
- Line 4 (return a / b) – Syntax Error. Missing semicolon after the return statement.
- Line 9 (float num = 15.5) – Syntax Error. Missing semicolon after the declaration.

- Line 15 (printf("Result is 5\n")) – Syntax Error. Missing semicolon after printf().
- Line 12 (result = divide(10);) – Semantic Error. Function is called with an incorrect number of arguments.
- Line 14 (if(result = 5)) – Intermediate Code Issue (Logic Error). Assignment operator (=) is used instead of comparison (==), producing incorrect intermediate code.
- Line 29 (total = x + y * result + 0;) – Code Optimization Issue. The addition of 0 is redundant and can be removed during optimization.
- Line 20 (int z = x / y;) – Runtime Error. Division by zero.
- Line 23 (arr[4]) – Runtime Error. Array index exceeds its valid range.
- Line 26 (*ptr = 25;) – Runtime Error. Dereferencing an uninitialized pointer.
- Line 31–33 (while(x < 20)) – Runtime/Logical Error. Infinite loop because x is never incremented.

(C) Compilation vs Execution:

(i) Errors Detected at Compile Time

- Invalid identifier (#value)
- Missing semicolon after return
- Missing semicolon after float num
- Missing semicolon after printf
- Incorrect number of function arguments (divide(10))

(ii) Errors Detected During Execution

- Division by zero (x / y)
- Array out-of-bounds access (arr[4])
- Dereferencing an uninitialized pointer (*ptr)
- Infinite loop (while(x < 20))

(iii) Why Some Errors Prevent Further Compilation

Lexical and syntax errors occur during the early phases of compilation. If the compiler cannot recognize valid tokens or parse the program correctly, it stops compilation and cannot proceed to semantic analysis, code generation, or optimization. Therefore, runtime errors are detected only after compilation succeeds.

(D) correct program:

```
#include <stdio.h>
```

```
int divide(int a, int b)
{    return a / b;
}

int main() {int value = 20;    float num
= 15.5;    int result;    result =
divide(10, 2);    if(result == 5)
{printf("Result is 5\n");}    int x = 10;
int y = 2;    int z = x / y;    int arr[4] =
{1,2,3,4};    printf("%d\n", arr[3]);
int data = 25;    int *ptr = &data;
*ptr = 25;    int total;    total = x + y *
result;    while(x < 20)
{printf("%d\n", x);        x++;}
printf("Total = %d\n", total);
printf("Pointer Value = %d\n", *ptr);
printf("Division Result = %d\n", z);
return 0;
}
```

(E) Deep Analysis (Evaluation)

(i) Effect of Incorrect Pointer Usage:

If a pointer is declared but not initialized, it points to an unknown memory location. Writing through such a pointer results in undefined behavior, often causing a segmentation fault or program crash.

(ii) Impact of Array Bounds Violation:

Accessing an array element beyond its valid range reads or writes memory that does not belong to the array. This may produce incorrect values, corrupt memory, or crash the program.

(iii) Effect of Operator/Expression Errors on Intermediate Code:

Using the assignment operator (=) instead of the comparison operator (==) changes the generated intermediate code. Instead of checking whether result equals 5, the compiler generates code that assigns 5 to result, making the condition always evaluate as true. This changes the program's logic and produces incorrect execution.

(F) Advanced Task (Create Level) (i)

Improvements in Compiler Design A.

Detect Runtime Errors Earlier:

- Use static analysis to detect possible division by zero, null pointer dereferencing, and array index violations before execution.
- Perform data flow analysis to identify uninitialized variables and pointers during compilation. B.

  Improve Debugging Messages:
- Display the exact line and column number where the error occurs.
- Provide meaningful error messages with suggested corrections (e.g., *"Did you mean '==' instead of '='?"*).

(ii) Optimization Phase Improvements:

The optimization phase can eliminate redundant expressions to improve efficiency. For example:

total = x + y * result + 0;

can be optimized to:

total = x + y * result;